A Preliminary Study on the Vocabulary of Flaky Tests in Swift

João Medeiros
Federal University of Pernambuco
Recife, Brazil
jpcm2@cin.ufpe.br

Breno Miranda
Federal University of Pernambuco
Recife, Brazil
bafm@cin.ufpe.br

Abstract

Flaky tests yield inconsistent results without any code changes, damaging confidence in the test suites. This work investigates the feasibility of detecting flaky tests in the Swift programming language through static analysis techniques combined with machine learning. From 14 open-source projects, we collected flaky tests through systematic re-execution and commit analysis, extracted tokens, and trained five classifiers (Random Forest, Decision Tree, Naive Bayes, SVM, and KNN) with TF-IDF (Term Frequency-Inverse Document Frequency) vectors. The Random Forest classifier achieved the best results, followed by SVM, demonstrating that it is possible to predict instability with high accuracy based solely on code. The results confirm the effectiveness of static detection and vocabulary as predictors of flakiness within the Swift ecosystem, helping developers identify these issues proactively.

Keywords

Software testing, Flakiness, Swift, iOS development

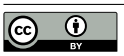
ACM Reference Format:
João Medeiros and Breno Miranda. 2026. A Preliminary Study on the Vocabulary of Flaky Tests in Swift. In *3rd International Flaky Tests Workshop 2026 (FTW '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3786178.3788328

1 Introduction

Although they do not directly affect software functionality, *flaky tests* represent a considerable consumption of resources in deployment systems [2–4]. Micco [4] revealed that 16% of the 4.2 million tests evaluated in 2017 at Google exhibited unstable behavior, causing recurring delays in releases.

Given this scenario, machine learning techniques have been applied to the detection of unstable tests, with several studies considering the presence of specific identifiers in test code and other general test characteristics as potential predictors of instability [1, 7]. Pinto et al. [5] evaluated machine learning algorithms to predict unstable tests from the source code of Java applications. To this end, 64 thousand tests were executed 100 times, building a dataset of stable and unstable tests. Using features such as identifier token count, models like Random Forest and SVM achieved an F-score of 0.95, demonstrating the effectiveness of static detection.

This work investigates the feasibility of detecting flaky tests in the Swift programming language through static analysis techniques combined with machine learning. The adopted methodology is divided into three main stages: first, the empirical identification of flaky tests through the systematic re-execution of test suites in open-source Swift projects; second, the mining of open-source repositories to identify flaky tests from the history of pull requests and commits; and finally, the application of classification algorithms on features statically extracted from the test code, with the objective of training models capable of predicting whether a test is flaky without the need for execution.



This work is licensed under a Creative Commons Attribution 4.0 International License.
FTW '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2409-1/26/04
https://doi.org/10.1145/3786178.3788328

2 Methodology

This work investigated two main research questions: the accuracy in predicting unstable test cases based on identifiers present in Swift test source code (RQ1) and which identifiers are strongly associated with instability (RQ2). The utilized pipeline is visually exemplified in Figure 1.

Collection of test cases. The collection of test cases was carried out in a hybrid manner in open source repositories. The projects considered in our study are shown in Table1. First, through local re-execution, where tests that failed only in some repetitions were classified as flaky. Second, through the analysis of commits and pull requests, searching for the keywords *"flaky"* and *"flaky test"* in the repositories, applying filters to confirm that they were fixes for unstable tests, and selecting the version prior to the fix. Tests not classified as flaky were considered stable.

Data Treatment. In data treatment, tests without assertion elements from the XCTest library were removed. The code was split into tokens with the removal of stop words and common language terms that do not contribute to semantic differentiation (such as *var*, *let*, *func*, *class*), allowing a simplified representation for the identification of lexical patterns associated with instability.

Classification Model. For classification, five machine learning algorithms were used: Random Forest, Decision Tree, Naive Bayes, SVM, and KNN Pinto et al. [5]. The models were trained with TF-IDF vectors and validated by stratified K-Fold cross-validation (k=5), preserving the class proportion. Performance was measured using precision, recall, F1-score, MCC, and AUC, with an emphasis on the weighted F1-Score, which combines precision and recall, prioritizing the correct identification of unstable tests [5, 6].

3 Results

Answer to RQ1. Based on the chosen models for the classifier, the evaluations were performed using stratified cross-validation with 5 folds, demonstrating robust performance in the detection of unstable tests, as shown in Table 2. These results indicate that the models are capable of providing assertive predictability of flaky tests based on attributes extracted statically from the source code. This approach corroborates previous research that utilized the identification of characteristic instability patterns, such as execution

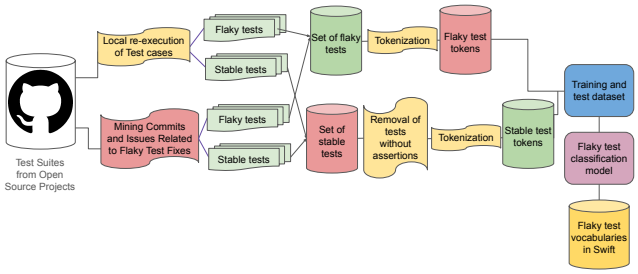


Figure 1: Process for building the flakiness vocabulary

Table 1: Flaky (F) and Non-Flaky (NF) Test Counts in Swift

Project	F	NF	Project	F	NF
Alamofire	3	554	Nuke	11	126
DuckDuckGo iOS	17	1003	sentry-cocoa	3	5041
firefox-ios	9	4487	Signal-iOS	7	713
GRDB.swift	9	172	swift-comp.-architecture	4	686
ios-oss	1	1235	swift-package-manager	9	1173
Kingfisher	3	703	swift-nio	10	2657
Maccy	2	32	vapor	3	3767
			Total	**91**	**20446**

order dependence, use of global variables, asynchronous functions, and other typical indicators. These findings are consistent with the work of Pinto et al. [5], who demonstrated that models based on Random Forest and SVM tend to exhibit greater robustness in this scenario, consolidating themselves as effective alternatives for the automated detection of non-deterministic tests.

Table 2: Classifier Results

Algorithm	Precision	Recall	F1	MCC	AUC
Random Forest	0.92	0.83	0.87	0.77	0.97
Decision Tree	0.80	0.86	0.82	0.66	0.82
Naive Bayes	0.87	0.83	0.85	0.71	0.90
Support Vector Machine	0.87	0.89	0.87	0.75	0.93
K-Nearest Neighbours	0.84	0.79	0.81	0.64	0.89

Answer to RQ1: Overall, all classifiers performed well based on the utilized test set. In particular, Random Forest was the classifier with the best result found.

Answer to RQ2. Table 3 presents the 5 tokens with the highest information gain (Mutual Information), which proved to be the most relevant for distinguishing unstable tests (flaky tests) from stable tests in the analyzed set. Information gain reflects how much the presence of a specific token contributes to reducing uncertainty about a test's class, thereby indicating syntactic or semantic patterns strongly associated with the occurrence of flakiness. Overall, the most informative tokens are concentrated in four main categories associated with flakiness: (i) concurrency and asynchronous execution (*async*, *await*); (ii) state or persistence dependence (*in commit*); and (iii) temporal dependence. These results corroborate

Table 3: The 5 features with the best information gain

Token	*in commit*	*xctasserтequal*	*in*	*async*	*await*
Info. Gain	0.175	0.172	0.166	0.145	0.140

the hypothesis that instability in Swift tests is strongly related to the presence of constructs that introduce non-determinism, environment dependence, and runtime variation.

Answer to RQ2: The information gain analysis showed that tokens related to concurrency and asynchronous execution, state manipulation, and interface interaction are strongly associated with flakiness in Swift tests.

4 Conclusion

This work investigated the feasibility of predicting test instability in Swift using only the source code vocabulary, without the need for execution. From 14 open-source projects, collected through a hybrid methodology (systematic re-execution and commit analysis), the Random Forest model achieved an F1-Score of 0.87 and an AUC of 0.97. The validation of the hybrid approach, testing only local re-execution data, maintained an F1-Score between 80-90%, confirming the methodological robustness. We identified that the tokens most correlated with instability are concentrated in constructs related to concurrency (*async*, *await*), state dependence (*in commit*, *writefilecontents*), temporal dependence (*now*, *timespec*), and UI interaction (*accessibilityidentifiers*). This unstable vocabulary in Swift reflects the root causes of non-determinism documented in the literature, adapted to the iOS/macOS ecosystem. The results provide the first empirical evidence on vocabulary-based flaky test detection in Swift, suggesting a promising path for the development of static analysis tools that alert developers about problematic tests before they reach Continuous Integration, saving time and resources in the development cycle.

Acknowledgments

This research was partially supported by CNPq through grants 315765/2023-2 and 408651/2023-7.

References

[1] Abdulrahman Alshammari, Christopher Morris, Michael Hilton, and Jonathan Bell. 2021. FlakeFlagger: Predicting Flakiness Without Rerunning Tests. In *Proceedings of the 43rd International Conference on Software Engineering* (Madrid, Spain) *(ICSE '21)*. IEEE Press, 1572–1584.

[2] Kim Herzig and Nachiappan Nagappan. 2015. Empirically detecting false test alarms using association rules. In *Proceedings of the 37th International Conference on Software Engineering - Volume 2* (Florence, Italy) *(ICSE '15)*. IEEE Press, 39–48.

[3] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *Proceedings of the 22nd International Symposium on Foundations of Software Engineering* (Hong Kong, China) *(FSE 2014)*. 643–653.

[4] John Micco. 2017. The State of Continuous Integration Testing @Google.

[5] Gustavo Pinto, Breno Miranda, Supun Dissanayake, Marcelo d'Amorim, Christoph Treude, and Antonia Bertolino. 2020. What is the Vocabulary of Flaky Tests?. In *Proceedings of the 17th International Conference on Mining Software Repositories* (Seoul, Republic of Korea) *(MSR '20)*. Association for Computing Machinery, New York, NY, USA, 492–502. doi:10.1145/3379597.3387482

[6] Rafael Rampim Soratto and Marco Aurélio Graciotto Silva. 2023. Vocabulary of Flaky Tests in Javascript. In *Proceedings of the XXII Brazilian Symposium on Software Quality* (Brasília, Brazil) *(SBQS '23)*. New York, NY, USA, 168–177.

[7] Roberto Verdecchia, Emilio Cruciani, Breno Miranda, and Antonia Bertolino. 2021. Know You Neighbor: Fast Static Prediction of Test Flakiness. *IEEE Access* 9 (2021). doi:10.1109/ACCESS.2021.3082424